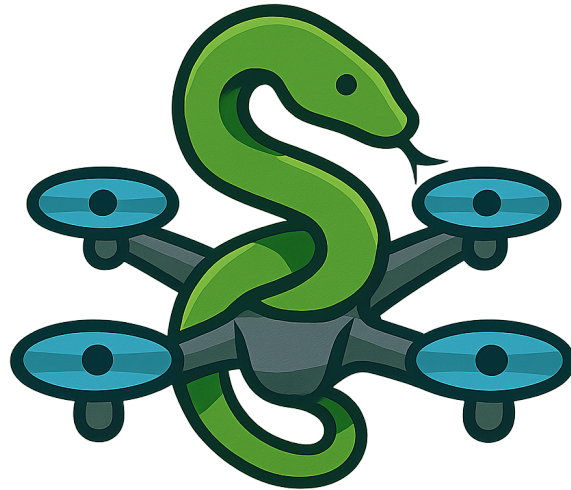


Technische Hochschule Ostwestfalen-Lippe

Bachelor of Science, Data Science

Gamification



Ausarbeitung zum Thema:

Vom Prototyp zur Spielschleife

Gamifizierung von *PythonWithDrones*

Erstellt von:	Niclas Falke, Stefan Reitemeyer und Jonas Pottmeier
Matrikelnummer:	20122542 (Niclas), 20124913 (Stefan) und 20116026 (Jonas)
Fachsemester:	4
Betrachteter Zeitraum:	10.04.2026 bis 26.07.2026
Abgabedatum:	26.07.2026
Prüfer:	Prof. Dr.-Ing. Rainer Rasche

Inhaltsverzeichnis

1	Abstrakt	1
2	Einleitung	1
3	Ausgangslage	2
3.1	Der Stand zum Modulwechsel	2
3.2	Warum das als Spiel noch nicht trug	2
4	Begriffe und Leitplanken	2
4.1	Gamifizierung	2
4.2	Mechanik, Dynamik und Erfahrung	3
4.3	Drei Grundbedürfnisse	3
4.4	Passung von Anforderung und Fähigkeit	3
4.5	Zwei Leitplanken	3
5	Umsetzung	4
5.1	Überblick	4
5.2	Mehr Verben für die Drohne	5
5.2.1	Neue Befehle	5
5.2.2	Lösungsbedingungen	6
5.3	Aufgabenkurve	6
5.4	Auswertung eines Laufs	7
5.4.1	Kennzahlen	7
5.4.2	Abschlussanzeige	7
5.4.3	Terminal	7
5.5	Bestenliste	8
5.5.1	Architektur	8
5.5.2	Was gewertet wird	8
5.5.3	Betrieb ohne Backend	9
5.6	Level-Editor	9
5.6.1	Aufbau	9
5.6.2	Vom Entwurf zum Level	10
5.6.3	Einordnung	10
5.7	Reibungsabbau	11
6	Auswertung und Diskussion	12
6.1	Zuordnung zu den Grundbedürfnissen	12
6.2	Reichweite der Aussagen	12
6.3	Bewusst getroffene Festlegungen	12
6.4	Worauf die Wertung zeigt	13
7	Schlussfolgerung	13
7.1	Zusammenfassung	13
7.2	Nächste Schritte	13
	Literatur	15

1 Abstrakt

PythonWithDrones ist eine Lernanwendung, in der Python-Quelltext eine virtuelle Drohne durch dreidimensionale Level steuert. Zum Abschluss des Vorgängermoduls lag ein lauffähiger Prototyp vor, dessen Ablauf mit dem Erreichen des Zielportals endete [1]. Das Level galt damit als gelöst, und die Interaktion war beendet. Die vorliegende Arbeit beschreibt, wie diese Anwendung zwischen dem 10. April und dem 26. Juli 2026 zu einem Spiel mit geschlossener Schleife ausgebaut wurde. Hinzugekommen sind ein erweiterter Handlungsraum der Drohne mit Schieben, Aufnehmen, Abliefern und Vorausschauen, eine auf dreizehn Level verlängerte Aufgabenkurve, eine Auswertung jedes Laufs nach Abstürzen, geflogenen Feldern und Codezeilen, eine serverseitige Bestenliste mit Kontoverwaltung und Versuchsverlauf sowie ein dreidimensionaler Level-Editor, mit dem Spielende eigene Aufgaben bauen und einreichen können. Die Auswertung ordnet die gebauten Elemente den drei Grundbedürfnissen Kompetenz, Autonomie und soziale Eingebundenheit zu und zeigt, wie die einzelnen Entwurfsentscheidungen ineinandergreifen.

2 Einleitung

Am Ende des vorangegangenen Moduls stand eine Anwendung, die technisch funktionierte. Python lief im Browser, die Drohne bewegte sich, Level ließen sich lösen. Beim Ausprobieren mit Kommilitonen fiel jedoch etwas auf, das sich mit einem Blick in den Quelltext bestätigen ließ. Die Anwendung kannte für jedes Level genau zwei Zustände, nämlich Zielportal erreicht oder nicht erreicht. Ob eine Lösung mit vier Zeilen oder mit vierzig auskam, ob sie auf Anhieb saß oder erst nach zwölf Abstürzen, blieb ohne Folgen. Wer das letzte Level abgeschlossen hatte, hatte keinen Anlass, die Seite noch einmal zu öffnen.

Diese Beobachtung ist der Ausgangspunkt der vorliegenden Arbeit. Sie stellt nicht die Frage, wie sich Python im Browser ausführen lässt, denn das war Gegenstand des Vorgängermoduls und ist gelöst. Sie fragt stattdessen, welche Spielelemente sich in eine bestehende Lernanwendung einbauen lassen, welche Wirkung von ihnen strukturell zu erwarten ist und welche Kosten sie im laufenden Betrieb verursachen. Gamifizierung wird dabei nicht als Aufsatz verstanden, den man einer fertigen Anwendung überstülpt, sondern als Eingriff in die Regeln. Wenn eine Bestenliste nach Schritten sortiert, verändert das, worüber Lernende beim Schreiben ihres Programms nachdenken.

Die Arbeit stützt sich auf den Quelltextstand des Projekts und auf seine Versionsgeschichte [2] sowie auf die Ausarbeitung des Vorgängermoduls, in der der Aufbau des Prototyps beschrieben ist [1]. Als Startpunkt dient das Tag v1.0.0 vom 10. April 2026, das den Stand zum Abschluss des Vorgängermoduls markiert. Alles, was danach entstanden ist, gehört in den Betrachtungsrahmen.

Abschnitt 3 fasst den Ausgangsstand so weit zusammen, wie es zum Verständnis der späteren Eingriffe nötig ist. Abschnitt 4 legt die Begriffe und die beiden Leitplanken fest, an denen die Entwurfsentscheidungen anschließend gemessen werden. Abschnitt 5 bildet den Hauptteil und geht die sechs Arbeitsstränge durch, die seit April entstanden sind. Abschnitt 6 wertet das Ergebnis aus und ordnet die Elemente einem Bedürfnisraster zu. Abschnitt 7 fasst zusammen und beschreibt die nächsten Schritte.

3 Ausgangslage

3.1 Der Stand zum Modulwechsel

Die Anwendung ist ein statischer Export eines Next.js-Projekts [3] und wird über GitHub Pages ausgeliefert. Sie benötigte zu diesem Zeitpunkt keine serverseitige Komponente. Geschriebener Quelltext wird an einen Web Worker übergeben, in dem Pyodide einen zu WebAssembly übersetzten CPython-Interpreter betreibt [4]. Jeder Drohnenbefehl erzeugt dort ein Datenobjekt, das über `postMessage` an den Hauptthread zurückgereicht wird. Dort nimmt eine Warteschlange die Objekte entgegen und arbeitet sie nacheinander als Animationen in einer mit React Three Fiber aufgebauten Szene ab [5]. Der Fortschritt liegt im `localStorage` des Browsers. Der Aufbau dieses Prototyps ist in der Ausarbeitung des Vorgängermoduls ausführlich beschrieben [1].

Level sind YAML-Dateien. Sie beschreiben den Startpunkt der Drohne und eine Folge horizontaler Schichten, in denen jede Zelle über eine Block-Kennung belegt ist. Welche Eigenschaften eine solche Kennung hat, ob ein Block also eine Kollision auslöst oder als Ziel gilt, steht zentral in der Blockregistrierung. Sowohl die Python-Seite als auch die Darstellung greifen auf dieselbe Quelle zu.

Zum Zeitpunkt von `v1.0.0` umfasste dieser Aufbau sechs spielbare Level, neun Blocktypen und zehn öffentliche Methoden am Drohnenobjekt. Der Handlungsraum bestand aus Bewegen, Steigen, Sinken, Drehen und zwei Abfragen, mit denen sich ein blockierter Weg und das Erreichen des Portals prüfen ließen.

3.2 Warum das als Spiel noch nicht trug

Aus der Perspektive des Gamification-Moduls fehlten dieser Anordnung drei Dinge.

Erstens gab es keine Bewertung. Der einzige Wert, den die Anwendung über eine Lösung kannte, war ihre Existenz. Damit fehlte jede Grundlage für Rückmeldung, die über ein bloßes Geschäft hinausgeht, und damit auch jeder Maßstab für Verbesserung.

Zweitens gab es keinen Bezug zu anderen. Der Fortschritt lag ausschließlich lokal im Browser. Zwei Personen, die dasselbe Level lösten, erfuhren nichts voneinander, und ein Wechsel des Geräts löschte den Fortschritt.

Drittens gab es keinen Spielraum. Die Inhalte lagen als YAML-Dateien im Repository, und wer eigene Aufgaben beisteuern wollte, musste die Dateistruktur kennen, das Projekt auschecken und einen Pull Request stellen. Für die Zielgruppe, also Menschen, die gerade ihre erste Schleife schreiben, ist das keine erreichbare Handlung.

4 Begriffe und Leitplanken

Dieser Abschnitt legt fest, in welcher Bedeutung die zentralen Begriffe im weiteren Verlauf verwendet werden und woran die Entwurfsentscheidungen anschließend gemessen werden.

4.1 Gamifizierung

Gamifizierung meint in dieser Arbeit den Einsatz einzelner Elemente aus dem Spieldesign in einem Zusammenhang, der selbst kein Spiel ist. Entscheidend ist die Beschränkung auf Elemente. Es geht nicht darum, aus der Lernanwendung ein Spiel zu machen, sondern einzelne Bausteine zu übernehmen, deren Wirkung aus Spielen bekannt ist.

Die drei am häufigsten übernommenen Bausteine sind Punkte, Abzeichen und Bestenlisten. Punkte machen eine Leistung zählbar, Abzeichen machen sie sichtbar, Bestenlisten machen sie vergleichbar. Im vorliegenden Projekt sind Punkte in Form der Kennzahlen je Lauf und die Bestenliste umgesetzt, Abzeichen sind bewusst zurückgestellt (siehe Abschnitt 7.2).

4.2 Mechanik, Dynamik und Erfahrung

Beim Entwurf trennt diese Arbeit drei Ebenen. Die Mechanik ist die Regel, die im Quelltext tatsächlich geschrieben wird. Die Dynamik ist das Verhalten, das im Betrieb daraus entsteht. Die Erfahrung ist das, was dabei bei den Spielenden ankommt. Der Nutzen der Trennung liegt in der Richtung der Wirkung. Gebaut wird bei der Mechanik, erlebt wird bei der Erfahrung, und dazwischen liegt eine Schicht, die sich nur mittelbar festlegen lässt.

Ein Beispiel aus diesem Projekt macht das konkret. Die Mechanik ist die Sortierreihenfolge der Bestenliste. Die Dynamik ist, dass Spielende anfangen, ihren Lösungsweg zu kürzen, statt ihn nur lauffähig zu machen. Die Erfahrung ist das Gefühl, einen eleganten Algorithmus gefunden zu haben. Die Aufgabe des Entwurfs besteht darin, die Mechanik so zu wählen, dass die gewünschte Dynamik überhaupt entstehen kann.

4.3 Drei Grundbedürfnisse

Als Ordnungsrahmen für die Auswertung in Abschnitt 6 dienen drei Bedürfnisse, die eine Tätigkeit aus sich heraus tragen können.

- **Kompetenz** meint das Erleben, besser zu werden, und setzt voraus, dass ein Fortschritt abgestuft sichtbar ist und ein Fehlschlag eine benennbare Ursache hat.
- **Autonomie** meint den Spielraum, eigene Entscheidungen zu treffen, bis hin zur Wahl und zum Bau der eigenen Aufgabe.
- **Soziale Eingebundenheit** meint den Bezug zu anderen, sei es über einen Vergleich, über einen sichtbaren Namen oder über geteilte Inhalte.

Dieses Raster hat den praktischen Vorteil, dass es die Verteilung der gebauten Elemente sichtbar macht und damit zeigt, wo ein Entwurf breit aufgestellt ist.

4.4 Passung von Anforderung und Fähigkeit

Eine Aufgabe trägt dann, wenn ihre Anforderung und die Fähigkeit der Lösenden in einem engen Verhältnis stehen. Liegt die Anforderung darüber, entsteht Überforderung, liegt sie darunter, Langeweile. Für eine Lernanwendung mit fester Reihenfolge bedeutet das, dass die Steigerung zwischen zwei Levels weder zu klein noch zu groß sein darf und dass die Anwendung sichtbar machen sollte, was ein Level verlangt, bevor es geöffnet wird.

4.5 Zwei Leitplanken

Aus dem Zweck der Anwendung ergeben sich zwei Festlegungen, die für jedes eingebaute Spielelement gelten. Sie werden hier vorangestellt, weil in Abschnitt 5 mehrfach auf sie zurückgegriffen wird.

Erstens sind die gemessenen Größen Eigenschaften der Lösung und nicht Eigenschaften des Verhaltens, das zur Lösung geführt hat. Eine Wertung, die belohnt, wer am längsten Zeit hat oder am schnellsten tippt, misst nicht das, was die Anwendung vermitteln soll. Eine Wertung, die die Zahl der Schritte belohnt, misst genau das Verständnis des Levels.

Zweitens steuert keine Belohnung den Zugang zu Lerninhalten. Jedes Level bleibt ohne Konto, ohne Bestenliste und ohne Serverbetrieb vollständig spielbar. Ein Spielelement darf den Reiz einer Aufgabe erhöhen, es darf ihn aber nicht ersetzen und es darf niemanden aussperren.

5 Umsetzung

5.1 Überblick

Die Arbeit zwischen dem 10. April und dem 26. Juli 2026 lässt sich in sechs Stränge gliedern, die Tab. 1 zusammenfasst. Sie greifen ineinander, weil jeder von ihnen eine Voraussetzung für einen anderen schafft. Ohne den erweiterten Handlungsraum gäbe es keine Level, die mehr verlangen als einen Weg, und ohne Kennzahlen gäbe es nichts, was eine Bestenliste sortieren könnte.

Strang	Kern der Änderung	Zeitraum
Handlungsraum	Schieben, Aufnehmen, Abliefern, Vorschauen; fünf neue Blocktypen; Lösungsbedingungen im Levelformat	Juli 2026
Aufgabenkurve	Sechs auf dreizehn Level, überarbeitete Einstiegsreihe, Schlagworte als Vorschau	Juli 2026
Laufauswertung	Abstürze, geflogene Felder und Codezeilen je Lauf; Abschlussanzeige; Terminal für <code>print</code>	Juli 2026
Bestenliste	FastAPI-Dienst mit Postgres, Konten, JWT, Rangfolge und Versuchsverlauf	Juni bis Juli 2026
Level-Editor	Dreidimensionaler Editor, Testlauf, YAML-Export, Einreichung über GitHub	Mai bis Juli 2026
Reibungsabbau	Mobile Darstellung, eigene Fehlerseite, Rückmeldeknopf, automatisierte Tests	Juli 2026

Tab. 1: Die sechs Arbeitsstränge seit Version 1.0.0.

Gemeinsam ergeben diese Stränge eine geschlossene Schleife, die es vorher nicht gab. Abb. 1 zeigt sie im Überblick. Entscheidend ist der Rücklauf. Vor der Erweiterung endete der Ablauf beim Erreichen des Ziels, jetzt führt jede Auswertung wieder auf eine mögliche nächste Handlung.

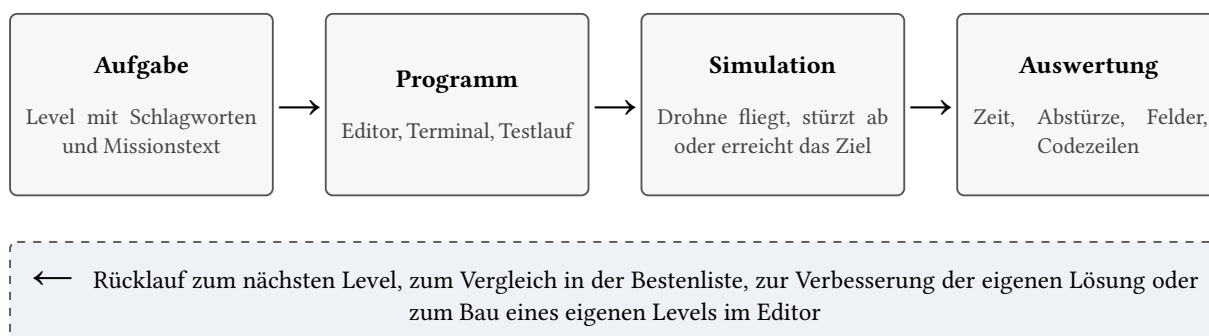


Abb. 1: Die Spielschleife nach der Erweiterung. Vor Version 1.0.0 endete der Ablauf hinter der Simulation, weil weder eine Auswertung noch ein Rücklauf existierten.

5.2 Mehr Verben für die Drohne

5.2.1 Neue Befehle

Der erste Eingriff betraf die Python-Schnittstelle selbst. Solange die Drohne nur fliegen und sich drehen konnte, gab es genau eine Art von Ziel, nämlich eine Position zu erreichen. Alle Level unterschieden sich damit nur in der Geometrie des Weges. Im Juli kamen vier Befehle hinzu, die den Zustand der Welt verändern oder abfragen.

- `drone.push()` schiebt einen als schiebbar gekennzeichneten Block um ein Feld weiter und rückt anschließend selbst nach. Ist das Feld dahinter belegt, bleibt die Aktion folgenlos und meldet den Grund.
- `drone.pickup()` und `drone.deliver()` nehmen ein Paket auf und legen es auf einer Ablagefläche wieder ab. Beide prüfen den Zustand und melden, wenn nichts aufzunehmen oder nichts abzulegen ist.
- `drone.scan(n)` liefert die Block-Kennungen der nächsten n Felder in Blickrichtung und bricht am ersten blockierenden Feld ab. Ohne Argument gibt die Methode eine einzelne Kennung zurück.

`scan` tritt dabei an die Stelle der früheren Abfrage `is_path_blocked`, die nur einen Wahrheitswert lieferte. Der Unterschied ist didaktisch bedeutsam. Eine Zeichenkette oder eine Liste zwingt dazu, das Ergebnis zu vergleichen und in eine Bedingung einzusetzen, während ein Wahrheitswert direkt in ein `if` fällt. Damit lassen sich Level bauen, in denen die Drohne unterscheiden muss, worauf sie trifft, statt nur ob sie auf etwas trifft.

Insgesamt wuchs die öffentliche Schnittstelle von zehn auf vierzehn Methoden, die Blockregistrierung von neun auf vierzehn Typen. Neu sind Münze, Kiste, Paket, Ablagefläche und Zielmarkierung. Abb. 2 zeigt ein Level, das drei dieser Bausteine gleichzeitig verwendet.

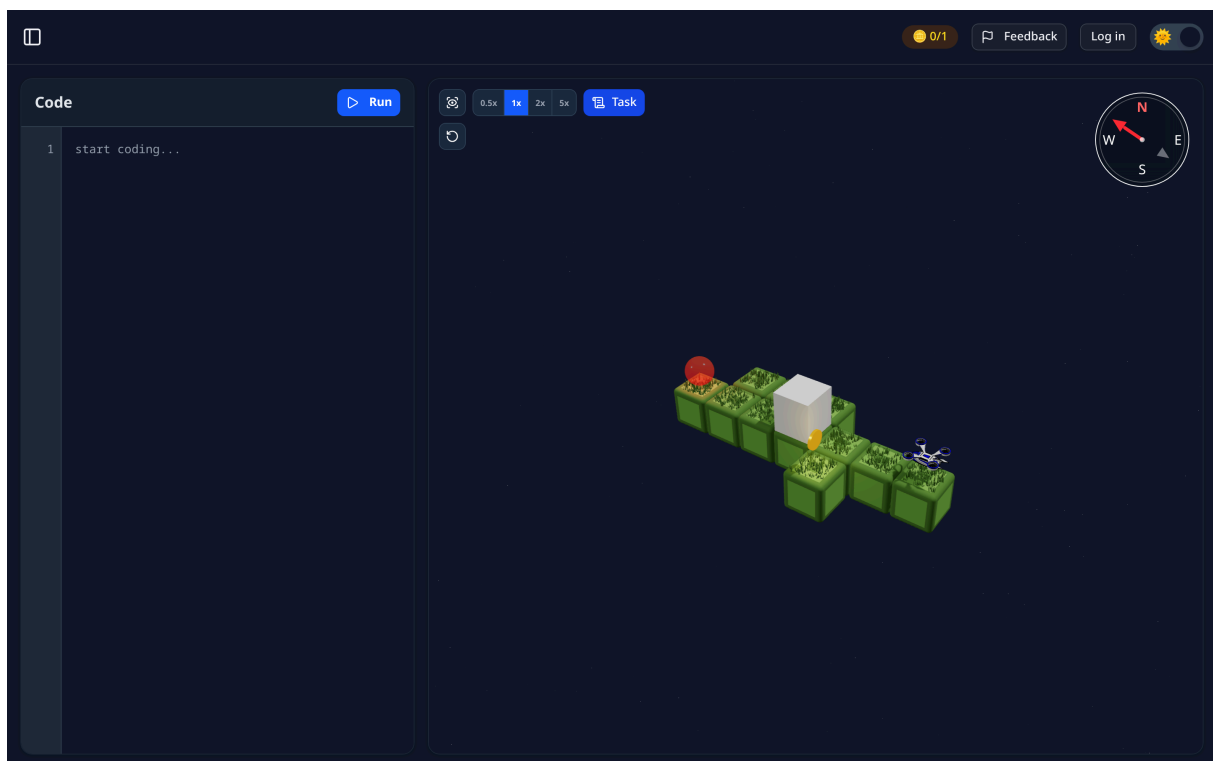


Abb. 2: Das Level *Push 'n Collect* vor dem ersten Lauf. Die Kiste verspermt den direkten Weg, die Münze liegt auf einem Seitenpfad, und der Zähler in der Kopfzeile hält den Stand der Lösungsbedingung fest.

5.2.2 Lösungsbedingungen

Damit die neuen Handlungen Bedeutung bekommen, wuchs das Levelformat mit. Ein Level kann nun neben dem Erreichen des Portals verlangen, dass eine bestimmte Zahl Münzen eingesammelt, ein Paket abgeliefert oder eine Kiste auf ein markiertes Feld geschoben wurde. Die Prüfung liegt im Levelmodell und läuft ab, sobald die Drohne das Portal betritt.

Der interessante Teil ist der Fall, in dem eine Bedingung offen ist. Hier sendet die Python-Seite eine eigene Nachricht vom Typ `hint`, deren Text die offenen Bedingungen benennt. Lernende erfahren damit unmittelbar, warum ein scheinbar erreichtes Ziel noch nicht zählt. Aus einem stummen Nichtereignis wird eine Erklärung.

5.3 Aufgabenkurve

Der Levelbestand stieg von sechs auf dreizehn. Ein Teil davon entstand aus einer Überarbeitung der Einstiegsreihe im Juli, bei der die ersten Level neu geordnet und ihre Beschreibungen gestrafft wurden. Die neuen Level greifen die erweiterten Befehle auf, sodass jeder Schritt der Reihe entweder ein Programmierkonzept oder eine Drohnenfähigkeit einführt, aber möglichst nicht beides gleichzeitig.

Jedes Level trägt Schlagworte, die in der Übersicht angezeigt werden. Sie nennen einerseits eine grobe Schwierigkeit und andererseits das behandelte Konzept, etwa Schleifen, Funktionen oder Bedingungen. Diese Vorschau setzt die in Abschnitt 4 beschriebene Passung zwischen Anforderung und Fähigkeit unmittelbar um, weil sie sichtbar wird, bevor jemand ein Level öffnet.

Gesperrte Level bleiben in der Übersicht sichtbar und tragen ein Schloss. Das ist eine bewusste Entscheidung gegen das Ausblenden, denn ein sichtbares, aber noch verschlossenes Ziel ist ein Anreiz, ein unsichtbares ist keiner.

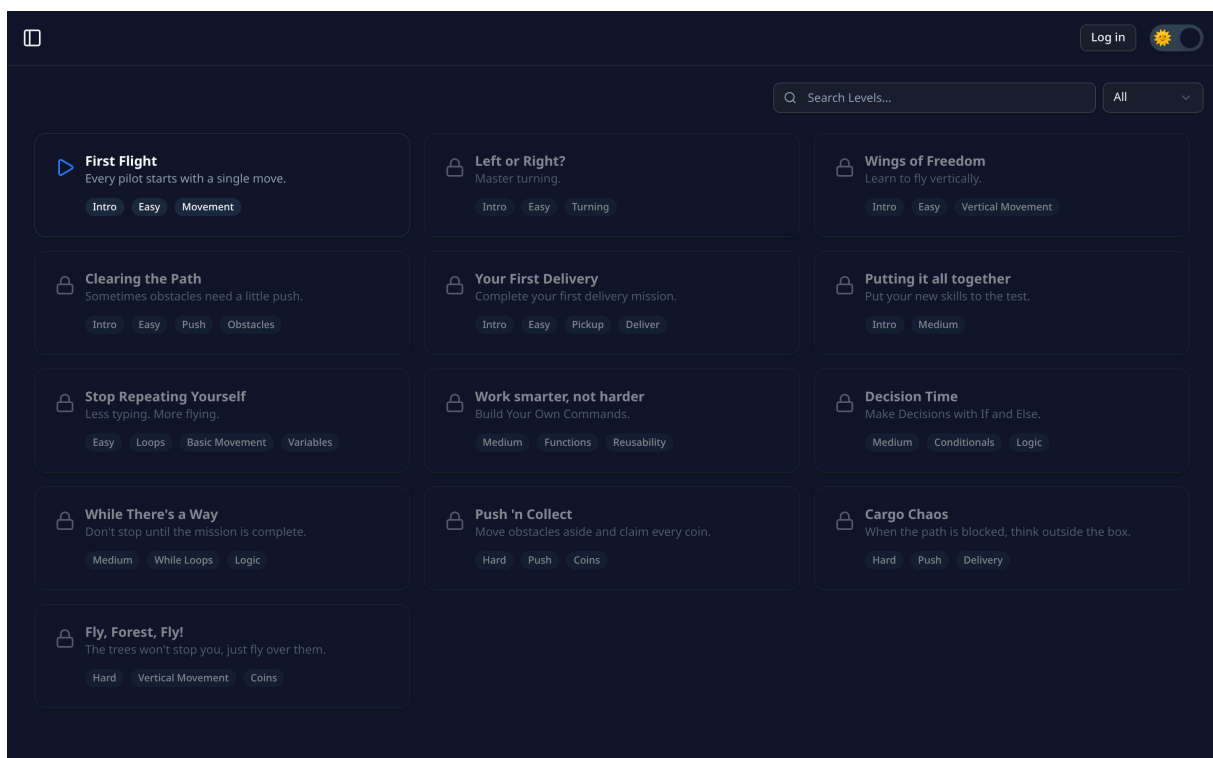


Abb. 3: Die Levelübersicht mit dreizehn Einträgen. Freigeschaltete Level tragen ein Abspielsymbol, gesperrte ein Schloss. Die Schlagworte künden Schwierigkeit und behandeltes Konzept an.

5.4 Auswertung eines Laufs

5.4.1 Kennzahlen

Die Python-Seite führt seit Juli zwei Zähler mit. `distance` steigt bei jedem erfolgreichen Ortswechsel, `crash_count` bei jeder Kollision. Beide werden über `get_stats()` an den Hauptthread gereicht. Die dritte Kennzahl, die Zahl der Codezeilen, ermittelt die Oberfläche selbst aus dem eingegebenen Text.

Diese Auswahl folgt der ersten Leitplanke aus Abschnitt 4.5. Alle drei Größen beschreiben das Programm und nicht die Person. Wie viele Felder ein Weg braucht, wie oft eine Lösung an eine Wand fliegt und wie kompakt sie formuliert ist, sind Eigenschaften des Algorithmus. Wie schnell jemand tippt, ist es nicht.

5.4.2 Abschlussanzeige

Beim Erreichen des Ziels erscheint eine Anzeige mit der benötigten Zeit und den drei Kennzahlen, ergänzt um einen kurzen Kommentar, der sich nach der Zahl der Abstürze richtet. Ein Lauf ohne Kollision wird anders quittiert als einer mit sieben. Der Kommentar ist bewusst knapp gehalten und übersetzt die nackte Zahl in eine Aussage.

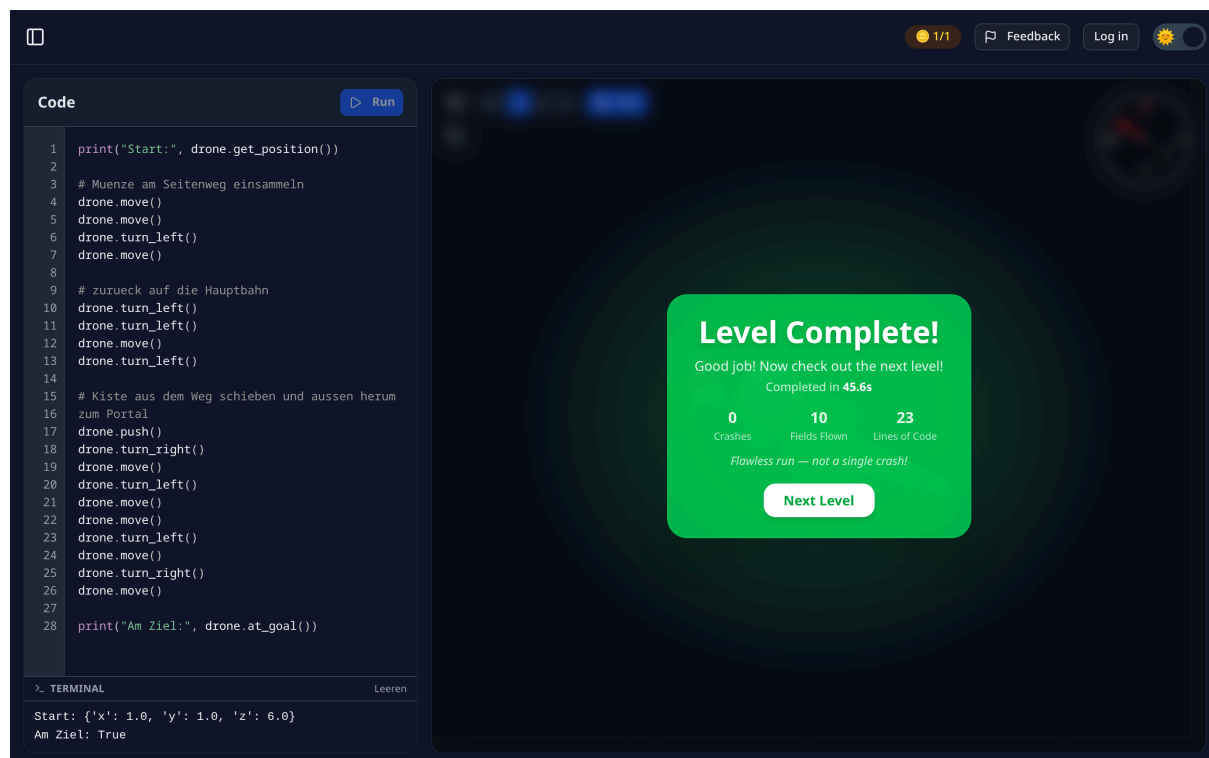


Abb. 4: Die Abschlussanzeige nach einem gelösten Level mit Zeit, Abstürzen, geflogenen Feldern und Codezeilen. Unten links zeigt das Terminal die Ausgaben der beiden `print`-Aufrufe.

5.4.3 Terminal

Bis Juli lief die Ausgabe von `print` ausschließlich in die Entwicklerkonsole des Browsers. Für die Zielgruppe war sie damit praktisch nicht vorhanden, obwohl `print` das erste Werkzeug ist, mit dem Anfänger den Zustand eines Programms untersuchen. Der Worker leitet die Ausgabe jetzt zusätzlich als eigene Nachricht an den Hauptthread weiter, wo sie unter dem Editor in einem Terminalbereich erscheint. Dieselbe Bahn nutzen auch die Hinweistexte der Drohnenbefehle, etwa wenn `pickup` ins Leere greift.

Der Beitrag zur Gamifizierung liegt hier nicht in einem Spielelement, sondern in der Verkürzung der Rückmeldeschleife. Ein Fehler, dessen Ursache sichtbar ist, führt zu einem weiteren Versuch. Ein Fehler ohne sichtbare Ursache führt zum Abbruch.

5.5 Bestenliste

5.5.1 Architektur

Die Bestenliste ist der einzige Teil des Projekts, der eine serverseitige Komponente verlangt. Sie besteht aus einem FastAPI-Dienst [6] und einer Postgres-Datenbank [7], die zusammen mit dem Frontend über Docker Compose gestartet werden. Der Datenbankzugriff läuft asynchron über SQLAlchemy [8]. Kennwörter werden mit Argon2 gehasht, die Anmeldung liefert ein JWT, und die Anmelde- und Registrierungsrouten sind ratenbegrenzt. Zugelassene Ursprünge stehen in einer Liste in der Umgebungsconfiguration.

Der Dienst führt drei Tabellen. `users` hält die Konten, wobei die Eindeutigkeit des Namens über einen funktionalen Index auf der kleingeschriebenen Fassung erzwungen wird, sodass sich zwei Konten nicht allein durch Groß- und Kleinschreibung unterscheiden können. `scores` hält je Konto und Level genau einen Eintrag, nämlich die erste Lösung. `attempts` hält jeden einzelnen Versuch.

5.5.2 Was gewertet wird

Die Rangfolge ergibt sich zuerst aus der Zahl der Schritte, dann aus der benötigten Zeit und zuletzt aus der Zahl der Codezeilen. Diese Reihenfolge ist die inhaltlich wichtigste Entscheidung des Strangs. Hätte die Zeit an erster Stelle gestanden, wäre die Bestenliste eine Tippgeschwindigkeitsliste geworden, und wer die Lösung bereits kennt, hätte immer gewonnen. Die Schrittzahl dagegen ist eine Eigenschaft des gefundenen Weges. Sie lässt sich nur verbessern, indem man das Level besser versteht.

Gewertet wird die erste Lösung eines Levels. Das hält die Liste als Vergleich der Lösungsqualität zusammen, statt sie in einen Vergleich der verfügbaren Freizeit zu verwandeln. Ergänzend hält die Tabelle `attempts` den vollständigen Verlauf, den Angemeldete unter der eigenen Fortschrittsanzeige einsehen können. Der Vergleich mit anderen ist damit einmalig, der Vergleich mit sich selbst dauerhaft.

Die Zeitmessung beginnt beim Öffnen des Levels und endet bei der ersten erfolgreichen Lösung. Sie läuft als Wanduhr im Hauptthread und ist damit unabhängig davon, mit welcher Geschwindigkeit die Animation abgespielt wird oder wie lange der Python-Interpreter für die Ausführung braucht.

RANK	PLAYER	STEPS	TIME	LINES OF CODE	WHEN
1	Niclas	9	48.2s	11	26 Jul 2026, 21:19
2	Stefan	9	1:11.4s	14	26 Jul 2026, 21:19
3	Jonas	11	1:03.9s	9	26 Jul 2026, 21:19
4	Mira	12	2:35.2s	21	26 Jul 2026, 21:19
5	Tom	16	3:28.7s	26	26 Jul 2026, 21:19

Abb. 5: Die Bestenliste einer laufenden Instanz. Die Level lassen sich einzeln auswählen, die Spaltenköpfe sind sortierbar. Die Reihenfolge folgt zuerst der Schrittzahl, wie der Vergleich der ersten drei Ränge zeigt.

5.5.3 Betrieb ohne Backend

Die Anwendung bleibt weiterhin ohne Server benutzbar, weil sie als statischer Export auf GitHub Pages liegt. Der API-Client liest dazu eine einzige Umgebungsvariable. Ist sie nicht gesetzt, liefert jeder Aufruf `null` zurück, und sämtliche Aufrufe der Bestenliste laufen ins Leere, ohne einen Fehler zu erzeugen. Das Spiel funktioniert dann vollständig, nur eben ohne Vergleich.

Anmeldung ist an keiner Stelle Voraussetzung für das Spielen. Der Anmeldedialog bietet neben Registrierung und Anmeldung ausdrücklich einen Gastmodus an. Damit ist die zweite Leitplanke aus Abschnitt 4.5 eingehalten, denn kein Lerninhalt hängt an einer Belohnungsmechanik.

5.6 Level-Editor

5.6.1 Aufbau

Der Editor ist der umfangreichste einzelne Beitrag des Zeitraums. Er wurde im Mai begonnen und bis Juli in mehreren Durchgängen ausgebaut. Er läuft vollständig im Browser und arbeitet auf derselben Datenstruktur wie das Spiel selbst.

Die Oberfläche trennt zwei Modi. Im Setzmodus platziert oder löscht ein Klick einen Block auf der aktiven Ebene, im Kameramodus dreht und zoomt derselbe Klick die Ansicht. Da beim Bauen ständig zwischen beidem gewechselt wird, lässt sich der Kameramodus zusätzlich durch Halten der Alt-Taste vorübergehend aktivieren, das Löschen entsprechend über die X-Taste. Neben Setzen und Löschen gibt es ein Werkzeug für den Startpunkt der Drohne.

Die Ebenen werden über einen Schrittwähler durchlaufen. Sichtbar bleiben dabei alle Ebenen, die aktive wird durch ein Gitter hervorgehoben, sodass die Tiefe beim Bauen erhalten bleibt. Ein eigenes Formular legt die Ausdehnung in allen drei Achsen fest, ein weiteres Titel, Beschreibung und Schlagworte.

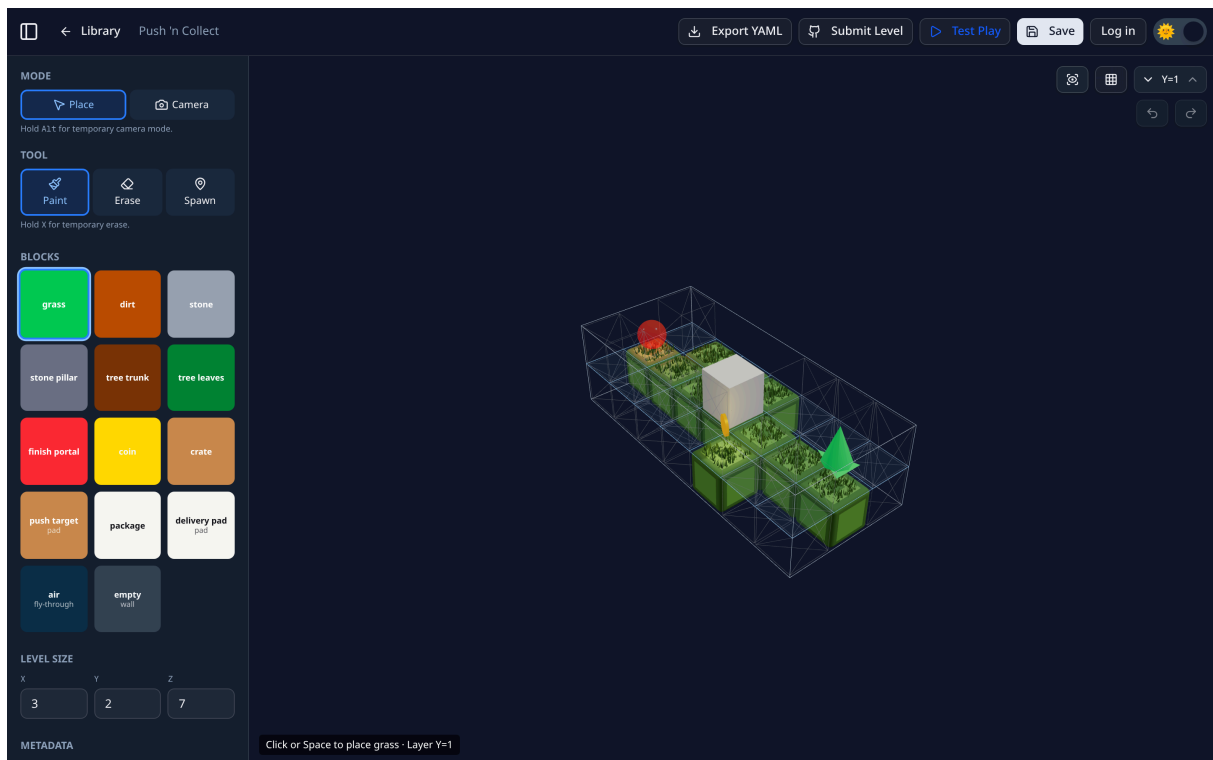


Abb. 6: Der Level-Editor mit einem geladenen Level. Links Modus, Werkzeuge, Blockpalette und Größenangaben, oben rechts der Ebenenwähler und die Rücknahme, oben die Aktionen Export, Einreichung, Testlauf und Speichern.

5.6.2 Vom Entwurf zum Level

Drei Wege führen aus dem Editor heraus. Der Testlauf öffnet das gebaute Level in einem Dialog mit vollständiger Spielumgebung, also eigenem Editor, eigener Pyodide-Instanz [4] und eigener Szene. Der Dialog baut diese Umgebung erst beim Öffnen auf und wirft sie beim Schließen wieder weg, damit nicht dauerhaft ein zweiter Interpreter mitläuft. Der Export schreibt eine YAML-Datei in den Download-Ordner. Die Einreichung öffnet ein vorbereitetes GitHub-Issue, in dessen Formular das erzeugte YAML bereits eingetragen ist.

Eine Entscheidung aus dem Juli verdient eigene Erwähnung. Ursprünglich mussten Autoren die Lösungsbedingungen eines Levels von Hand angeben, also etwa eintragen, dass drei Münzen einzusammeln sind. Seitdem leitet der Editor die Bedingungen aus dem gebauten Level ab. Wer drei Münzen platziert, baut ein Level mit drei einzusammelnden Münzen. Damit können Geometrie und Bedingung nicht mehr auseinanderlaufen, und der Bauvorgang wird um einen Schritt kürzer.

5.6.3 Einordnung

Der Editor adressiert das Bedürfnis nach Autonomie, das von Punkten und Bestenlisten gerade nicht bedient wird. Er verschiebt die Rolle vom Lösen vorgegebener Aufgaben zum Stellen eigener. Für den Lerneffekt ist das insofern bedeutsam, als das Bauen eines lösbaren Levels verlangt, die Regeln der Simulation vollständig verstanden zu haben.

Gleichzeitig ist er eine Inhaltsquelle. Der Aufwand für ein neues Level lag vorher bei handgeschriebenem YAML und mehreren Stellen im Quelltext, die synchron gehalten werden müssen. Er liegt jetzt bei einem Formular und einem Klick auf Einreichen.

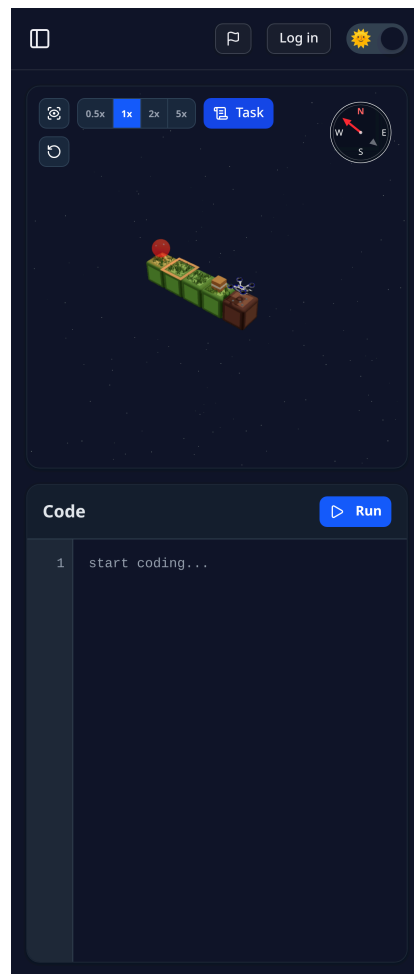


Abb. 7: Die Levelansicht auf einem schmalen Bildschirm. Szene und Editor stehen untereinander statt nebeneinander, der Aufgabentext lässt sich über die Schaltfläche Task ein- und ausblenden.

5.7 Reibungsabbau

Der sechste Strang enthält Arbeiten, die für sich genommen keine Spielelemente sind, aber Voraussetzung dafür, dass die anderen fünf wirken.

Im Juli wurde die Darstellung auf schmalen Bildschirmen überarbeitet. Betroffen waren unter anderem der Ebenenwähler des Editors, der außerhalb des sichtbaren Bereichs lag, ein Mausevents, das nie gebunden wurde, und eine überfüllte Kopfzeile. Da ein erheblicher Teil der Zugriffe von Mobilgeräten kommt, entscheidet diese Ebene darüber, ob jemand überhaupt bis zum ersten Level kommt.

Ein zweiter Punkt betrifft ungültige Adressen. Seit Juli existiert eine eigene Fehlerseite mit animierter Drohne, und die Routenkonfiguration weist unbekannte Parameter sauber ab. Ein Aufruf von `/level/99` landet damit auf der gestalteten Seite des Spiels statt auf einer Fehlermeldung des Servers. Das ist kein Spielelement, verhindert aber, dass ein Fehlgriff wie ein Defekt der Anwendung aussieht.

Schließlich entstand im Juli eine automatisierte Testabdeckung. 118 Testfunktionen prüfen die Spiellogik in `game.py` und `model.py` sowie den Bestenlisten-Dienst. Die Spieltests täuschen dazu das Brückenmodul zur JavaScript-Seite vor, sodass der unveränderte Quelltext unter normalem CPython läuft. Die Dienst-Tests sprechen die Anwendung im selben Prozess an und arbeiten je

Test gegen eine wegwerfbare SQLite-Datenbank statt gegen Postgres. Damit ist die Spiellogik, die im Browser sonst nur schwer zu prüfen wäre, in einer gewöhnlichen Testumgebung abgesichert.

6 Auswertung und Diskussion

6.1 Zuordnung zu den Grundbedürfnissen

Tab. 2 ordnet die gebauten Elemente den drei Grundbedürfnissen zu. Die Zuordnung ist als Entwurfsargument zu lesen.

Bedürfnis	Umgesetzte Elemente	Begründung
Kompetenz	Kennzahlen je Lauf, Abschlussanzeige, Versuchsverlauf, gestufte Levelreihe, Terminal, Hinweise bei offenen Bedingungen	Der Fortschritt wird abgestuft statt binär sichtbar. Fehlschläge liefern eine benennbare Ursache statt eines stummen Abbruchs.
Autonomie	Level-Editor mit Testlauf, Export und Einreichung; freie Wahl unter den freigeschalteten Levels; Gastmodus ohne Konto	Spielende können eigene Inhalte erzeugen und weitergeben, statt nur vorgegebene abzuarbeiten. Kein Lerninhalt hängt an einem Konto.
Soziale Eingebundenheit	Bestenliste je Level, Konten mit sichtbarem Namen, Einreichung eigener Level über das Repository	Der bislang rein lokale Fortschritt bekommt einen Bezugsrahmen und wird für andere sichtbar.

Tab. 2: Zuordnung der Elemente zu den drei Grundbedürfnissen aus Abschnitt 4.

Alle drei Spalten sind besetzt, und das ist das eigentliche Ergebnis der Zuordnung. Der Kompetenzstrang ist am dichtesten ausgebaut, weil er die unmittelbare Rückmeldung auf jeden Lauf trägt. Die Autonomie ruht auf dem Editor, der die Rolle der Spielenden von der Lösenden zur Aufgabenstellenden erweitert. Die soziale Ebene ruht auf der Bestenliste und den eingereichten Levels. Sie ist die jüngste der drei und bietet den größten Spielraum für die in Abschnitt 7.2 beschriebenen nächsten Schritte, etwa gemeinsame Aufgaben.

6.2 Reichweite der Aussagen

Die Wirkungsaussagen dieser Arbeit sind Entwurfsargumente, die sich aus dem gebauten System und aus den Leitplanken in Abschnitt 4 ergeben. Eine Nutzerstudie war im Zeitrahmen des Moduls nicht vorgesehen und ist als nächster Schritt in Abschnitt 7.2 beschrieben. Eine Erhebung mit tatsächlichen Anfängerinnen und Anfängern ist der geeignete Weg, um die hier begründeten Wirkungen zu prüfen, weil die Zielgruppe der Anwendung genau diese Gruppe ist.

6.3 Bewusst getroffene Festlegungen

Mehrere Entscheidungen im Entwurf hatten Alternativen, die aus guten Gründen nicht gewählt wurden. Sie werden hier zusammengefasst, weil sie den Rahmen abstecken, in dem das Ergebnis zu lesen ist.

Die Kennzahlen werden im Browser gezählt und vom Dienst übernommen. Das hält die Ausführung dort, wo sie hingehört, nämlich vollständig auf der Clientseite, und macht die Bestenliste ohne zusätzliche Rechenlast betreibbar. Für den Einsatz in einer Lehrveranstaltung ist dieses

Vertrauensmodell angemessen. Für einen offen zugänglichen Wettbewerb wäre eine serverseitige Nachrechnung der eingereichten Lösung der passende Ausbauschritt.

Die Bestenliste bringt eine serverseitige Komponente in ein Projekt, das ursprünglich ohne sie auskam. Die stille Abschaltung über eine einzige Umgebungsvariable löst das sauber, weil dieselbe Auslieferung sowohl mit als auch ohne Dienst vollständig spielbar bleibt. Die Anwendung gewinnt damit den Vergleich, ohne ihre Unabhängigkeit von einem Betrieb aufzugeben.

Im Editor gebaute Level liegen im `localStorage` des jeweiligen Browsers und verlassen ihn über eine YAML-Datei oder über ein vorbereitetes GitHub-Issue. Das genügt für die Übergabe an das Projektteam und hält den Editor frei von jeder Kontopflicht. Eine Ablage im Backend, mit der sich Level unmittelbar untereinander teilen lassen, ist der nächste Ausbauschritt und in Abschnitt 7.2 beschrieben.

6.4 Worauf die Wertung zeigt

Eine eingeführte Belohnung kann die ursprüngliche Motivation überlagern, wenn sie auf etwas anderes zeigt als auf das Lernziel. Genau das war beim Entwurf der Wertung der Prüfstein. Zwei Entscheidungen halten sie auf Kurs. Die Rangfolge nach Schritten belohnt das Verständnis des Levels, das ohnehin Lernziel ist, und lässt sich nicht durch bloßen Fleiß ersetzen. Und die Beschränkung auf die erste Lösung nimmt der Liste die Eigenschaft, durch Wiederholung erklimmbar zu sein. Die Belohnung zeigt damit auf dieselbe Fähigkeit, die die Anwendung vermitteln soll.

7 Schlussfolgerung

7.1 Zusammenfassung

Am Anfang dieses Moduls stand eine Anwendung, die Python im Browser ausführen und eine Drohne animieren konnte, deren Ablauf aber mit dem Erreichen des Ziels endete. Am Ende steht eine Anwendung mit einer geschlossenen Schleife. Ein Level stellt eine Aufgabe, die Simulation zeigt die Folgen des geschriebenen Programms, eine Auswertung bewertet die Lösung nach Schritten, Abstürzen und Codezeilen, eine Bestenliste stellt sie neben die anderer, und ein Editor erlaubt es, die nächste Aufgabe selbst zu bauen.

Der Umfang lässt sich am Ergebnis ablesen. Die spielbaren Level wuchsen von sechs auf dreizehn, die Drohnenbefehle von zehn auf vierzehn und die Blocktypen von neun auf vierzehn. Hinzu kamen ein Backend-Dienst mit Kontoverwaltung, ein dreidimensionaler Level-Editor und eine automatisierte Testabdeckung, die die Spiellogik außerhalb des Browsers absichert.

Die Auswertung zeigt, dass die gebauten Elemente alle drei Grundbedürfnisse erreichen und dabei den Leitplanken aus Abschnitt 4.5 folgen. Gemessen wird die Lösung und nicht die Person, und der Zugang zu den Lerninhalten hängt an keiner Belohnung.

7.2 Nächste Schritte

- **Nutzerstudie** mit Personen ohne Programmiererfahrung, mit einer Gruppe ohne und einer mit aktivierter Bestenliste, um die bislang begründeten Wirkungen zu prüfen.
- **Prüfung der Kennzahlen** durch eine serverseitige Plausibilitätskontrolle eingereicherter Werte, mindestens gegen eine untere Schranke der Schrittzahl je Level.
- **Level teilen** über eine Ablage der im Editor gebauten Level im Backend, sodass sie sich ohne GitHub-Konto veröffentlichen und von anderen spielen lassen.

- **Abzeichen für Konzepte** als Auszeichnungen, die nicht an Geschwindigkeit hängen, sondern daran, ein Level erstmals mit einer Schleife, einer Funktion oder einer Bedingung gelöst zu haben. Sie besetzen den bisher zurückgestellten dritten Baustein aus Abschnitt 4, ohne den Wettbewerbsdruck zu erhöhen.
- **Gemeinsame Aufgaben** in Form wöchentlich wechselnder Level oder Aufgaben, die von mehreren gelöst werden, um die soziale Ebene weiter auszubauen.
- **Zweite Wertung** neben der gewerteten Erstlösung, also eine gesonderte Bestenliste für die beste jemals eingereichte Lösung, damit sich auch das Überarbeiten eines Programms in der Rangfolge niederschlägt.

Literatur

- [1] N. Falke, S. Reitemeyer, und J. Pottmeier, „Python with Drones“, Ausarbeitung im Modul Software Design, 2026.
- [2] „PythonWithDrones, Quelltext-Repository“. [Online]. Verfügbar unter: <https://github.com/pottmeier/PythonWithDrones>
- [3] „Next.js Documentation“. [Online]. Verfügbar unter: <https://nextjs.org/docs>
- [4] „Pyodide Documentation“. [Online]. Verfügbar unter: <https://pyodide.org/>
- [5] „React Three Fiber Documentation“. [Online]. Verfügbar unter: <https://r3f.docs.pmnd.rs/>
- [6] „FastAPI Documentation“. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/>
- [7] „PostgreSQL Documentation“. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/>
- [8] „SQLAlchemy Documentation“. [Online]. Verfügbar unter: <https://docs.sqlalchemy.org/>